MITRE-CORE: Technical Architecture, Experiments & Interview Preparation

> **Canonical checkpoint**: `hgnn_checkpoints/network_v9_v3/network_it_best.pt` · **Primary metric**: AMI
> **Core claim**: Purely unsupervised alert-to-campaign correlation — zero labels required at inference
> **Verified on**: 6 public datasets + 1 real SOC deployment (May 2026)

Table of Contents

1. The Problem

A mature Security Operations Centre processes **500–10,000 alerts per day** from a heterogeneous stack of sensors: network IDS (Suricata, Snort), endpoint EDR (CrowdStrike, Defender), WAF (F5, Imperva), and SIEM aggregation. Each alert is an isolated event — "SSH brute force from 10.0.1.42", "lateral movement on HOST-07", "C2 beacon to 185.x.x.x". Individually, these alerts have poor fidelity and high false-positive rates.

The actual threat, however, is a **multi-stage attack campaign** — an adversary executing Reconnaissance → Initial Access → Lateral Movement → Exfiltration over hours or days, generating thousands of correlated alerts across multiple sensors. A skilled analyst reads these alerts and mentally correlates them into a narrative. MITRE-CORE automates that narrative construction.

Why Existing Approaches Fail

Approach	Failure Mode
Rule-based correlation (Splunk ES, QRadar)	Brittle — misses novel TTPs not in the ruleset; explosion of tuning required per environment

Approach	Failure Mode
Supervised ML (DeepCASE, WATSON)	Requires labelled campaign data — unavailable in most SOCs; doesn't generalise across environments
Simple clustering (K-Means on flow features)	Ignores structural relationships (shared IPs, temporal proximity, user paths) between alerts
Homogeneous GNN	Treats all entities identically — an IP node and a process node need fundamentally different learned representations

MITRE-CORE's answer: model alerts and the entities they involve (IPs, hosts, users, devices) as a *heterogeneous graph*, learn embeddings that capture structural attack patterns via self-supervised contrastive training, then cluster those embeddings into campaigns — *without any labels at inference time*.

---

2. What MITRE-CORE Is

MITRE-CORE is a **purely unsupervised** heterogeneous graph neural network (HGNN) pipeline for grouping raw security alerts into multi-stage attack campaigns. The word "purely" is load-bearing: no labels are consulted at inference time. The model generalises zero-shot to datasets it was never trained on.

The Three Inference Modes

MITRE-CORE supports three modes, in increasing label dependency:

Mode	Label Requirement	When to Use
Zero-shot (Unsupervised)	None — production default	Any new environment, zero configuration
Semi-supervised (SupCon)	Small labelled set for fine-tuning	When 100–1000 labelled campaigns are available
Supervised (Prototype)	Full campaign labels for prototype training	Benchmark / research comparison only

The primary contribution is zero-shot. Semi-supervised and supervised modes were implemented for ablation comparability. The key architectural claim — that graph topology provides signal beyond raw features, learned in a self-supervised manner — is validated entirely in the zero-shot regime.

Novel Technical Contributions

Contribution	Technical Substance
Heterogeneous graph schema	8+ node types, 29 edge relation types, each with independent GATConv weights
Topological NT-Xent loss	Graph edges define positive pairs — structurally superior to augmentation-based SimCLR

Contribution	Technical Substance
GAEC confidence scoring	HDBSCAN membership vectors replace softmax max-prob — calibrated to embedding geometry
Dataset-aware checkpoint routing	`dataset_profiler.py` selects checkpoint and clustering config from structural fingerprint
Soft-ZCA whitening	Decorrelates collapsed embeddings (cosine_sim > 0.95) without retraining
Single-layer architecture	Empirically validated: L=1 prevents over-smoothing on high-degree entity nodes
AMI as primary metric	Information-theoretic; robust to legitimate sub-clustering that ARI penalises

3. Verified Results

All numbers verified May 8, 2026. Seed=42 everywhere. AMI is primary (see §12).

Zero-Shot (Unsupervised) — network_v9_v3 Checkpoint

Dataset	AMI ↑	ARI ↑	Purity	Attack F1	n_clusters	Key Note
NSL-KDD	**0.673**	**0.752**	0.889	0.900	~10	Exceeds supervised (0.595)
UNSW-NB15	**0.582**	0.401	0.814	0.0	8	Spectral k=8; IXIA ceiling
TON_IoT	**0.717**	0.431	0.923	0.969	37	10K stratified sample; UMAP off
OpTC	**0.149**	0.048	0.999	0.942	25	binary_ARI = **1.000** (perfect sep)
SQTK_SIEM	**0.342**	0.184	0.562	0.0	~11	Real SOC data; ZCA required
CICIDS2017	—	0.284	—	—	—	Flow features only

HGNN vs Feature-Only Baselines (Unsupervised)

The most important validation: does the graph provide genuine signal beyond raw features?

Dataset	GMM on raw features	HGNN Zero-Shot	Lift
NSL-KDD	0.299	**0.752**	**+151% (2.5×)**
TON_IoT	0.233	**0.431**	+85%
UNSW-NB15	0.219	0.401	+83%

Interpretation: Even on NSL-KDD — a fully *disconnected* graph with zero IP or temporal edges — HGNN outperforms feature-only GMM by 2.5×. The benefit comes from the backbone's joint training on graph-rich datasets (UNSW-NB15, TON_IoT), which learns a representation space where attack families have geometric separation that a linear GMM cannot exploit.

Supervised Comparison (for reference only)

Dataset	Zero-Shot ARI	Supervised ARI	Verdict
NSL-KDD	**0.752**	0.595	Unsupervised **wins**
UNSW-NB15	**0.401**	0.497	Supervised marginal gain
TON_IoT	0.431	**0.845**	Labels essential for IoT
SQTK_SIEM	**0.184**	0.053	Prototype collapses on real data

4. End-to-End Pipeline

```
┌──────────────────────────────────────────────────────────────────────┐
│  INPUT: Raw alerts from any sensor (SIEM / IDS / EDR / WAF)          │
│  Formats: Splunk export, Wazuh JSON, Zeek/Bro logs, CSV, Parquet     │
└──────────────────────────┬───────────────────────────────────────────┘
                           │
                           ▼
┌──────────────────────────────────────────────────────────────────────┐
│  STEP 1 — INGESTION & NORMALISATION                                  │
│                                                                      │
│  MultiSourceIngestionPipeline (CS-4)                                 │
│  ├── Accepts N independent sensor feeds                              │
│  ├── Adds data_source column (e.g. "cisco_fw", "crowdstrike_edr")    │
│  └── Normalises to MITRE-format schema:                              │
│      [AlertId, timestamp, src_ip, dst_ip, hostname, username,        │
│       alert_type, tactic, campaign_id, protocol, service,            │
│       src_bytes, dst_bytes, stage, data_source]                      │
│                                                                      │
│  dataset_profiler.profile_dataset()                                  │
│  ├── Fingerprints structural properties (IP density, timestamps, hosts) │
│  └── Routes to optimal checkpoint + clustering config                │
└──────────────────────────┬───────────────────────────────────────────┘
                           │
                           ▼
┌──────────────────────────────────────────────────────────────────────┐
│  STEP 2 — HETEROGENEOUS GRAPH CONSTRUCTION                           │
│                                                                      │
│  AlertToGraphConverter.convert(df) → PyG HeteroData                  │
│                                                                      │
│  Nodes created:                                                      │
│    alert        [N_a,  6]   6-dim encoded features (see §5)          │
│    ip           [N_ip, 32]  hash-embedded IP address                 │
│    host         [N_h,  32]  hash-embedded hostname                   │
│    user         [N_u,  32]  hash-embedded username                   │
│    device       [N_d,  32]  IoT device (port-derived)                │
│    gateway      [N_g,  16]  subnet-derived gateway                   │
│    source_sensor[N_s,  32]  sensor origin node (CS-2)                │
│    process      [N_p,  32]  Linux process name (APT datasets)        │
│    command_line [N_c,  64]  commandline string (APT datasets)        │
│                                                                      │
│  Edges constructed (29 relation types — full catalogue in §5)        │
```

Checkpoint Routing Logic (`dataset_profiler.py`)

The profiler fingerprints the dataset from a 1,000-row sample (~0.1s) and selects the appropriate checkpoint:

```
n_unique_ips = max(src_ips.nunique(), dst_ips.nunique())
ip_density   = n_unique_ips / sample_size
has_timestamps = any col in ['timestamp', 'EndDate']
has_hostnames  = any col in ['hostname', 'process_name', 'CommandLine']

if n_unique_ips == 0 and not has_hostnames:
```

```
    → network_v9_v3   # NSL-KDD path: disconnected graph, feature-only
elif ip_density < 0.05 and has_timestamps:
    → network_v9_v3   # TON_IoT / UNSW: dense IoT / enterprise network
elif has_hostnames and n_unique_ips == 0:
    → multidomain_v2  # APT / host domain (OpTC, Linux_APT)
else:
    → network_v9_v3   # default for unrecognised network data
```

---

5. Graph Construction

5.1 The 6-Dimensional Alert Feature Vector

Every alert, regardless of source dataset, is normalised to a 6-dimensional integer feature vector before being fed to the model:

Dim	Feature	Encoding	Semantic Role
0	`tactic`	Integer 0–13 (MITRE ATT&CK phase)	Kill-chain position
1	`alert_type`	Integer hash 0–199	Signature / rule type
2	`hour`	Integer 0–23	Time-of-day attack pattern
3	`day_of_week`	Integer 0–6	Weekday / weekend pattern
4	`protocol`	Integer 0–255	Network protocol (TCP/UDP/ICMP)
5	`service`	Integer hash 0–49	Application layer (HTTP/SMB/RDP)

Why exactly 6 dims? A v9_v5 experiment tried 15-dim contextual features (IP frequency counts, temporal density ratios). It caused catastrophic regression: NSL-KDD ARI 0.714 → 0.003, TON_IoT 0.737 → 0.028. Root cause: contextual features computed from batch statistics at inference time, but the model was trained on a different distribution of those statistics. The 6-dim features are computed identically at training and inference time — completely stable across domain shifts.

5.2 Complete Edge Type Catalogue (29 Relations)

Alert-to-Alert (intra-type) — these run message passing directly between alerts:

Relation	Construction Logic	Semantics
`(alert, shares_ip, alert)`	GROUP BY src_ip or dst_ip	Alerts that touch the same IP address
`(alert, shares_host, alert)`	GROUP BY hostname / DeviceHostName	Alerts on the same machine
`(alert, temporal_near, alert)`	Sliding 1-hour window	Time-proximate alerts
`(alert, semantic_similar, alert)`	GROUP BY alert_type hash	Same signature type

Relation	Construction Logic	Semantics
`(alert, precedes, alert)`	Temporal adjacency within 2h (CS-3, disabled)	Kill-chain ordering

Cross-type bidirectional pairs — each has a forward and reverse direction:

Forward	Reverse	Semantics
`user → owns → alert`	`alert → owned_by → user`	User-alert association
`host → generates → alert`	`alert → generated_by → host`	Host-alert association
`ip → involved_in → alert`	`alert → involves → ip`	IP-alert association
`device → connects_via → gateway`	`gateway → connected_to → device`	IIoT topology
`sensor_type → classifies → device`	`device → classified_as → sensor_type`	Device category
`device → generates → alert`	`alert → generated_by → device`	IIoT event
`process → executes → alert`	`alert → executed_by → process`	Linux APT process
`command_line → associated_with → alert`	`alert → has → command_line`	APT commandline
`container → runs_in → pod`	`pod → runs → container`	Kubernetes topology
`process → spawned_in → container`	`container → spawns → process`	Container process
`ip → resolves_to → host`	`host → resolved_from → ip`	Bridge edge (ablated — zero effect)
`alert → collected_by → source_sensor`	`source_sensor → collects → alert`	CS-2: sensor origin

Total: 5 alert-alert + 24 cross-type = 29 edge relation types.

Each relation has **independent GATConv weights** — the model learns that `shares_ip` message passing should be weighted differently from `temporal_near`, which should differ from `user → owns`. This is impossible in a homogeneous GNN.

5.3 Why Heterogeneous Edges Matter

Consider two alerts from different attack stages that share a C2 IP address. In a homogeneous GNN they'd be connected by a generic edge and share gradient signal with every other alert connected to that IP. In the HGNN:

```
alert_A  ──(involves)──►  ip_185.x.x.x  ◄──(involves)──  alert_B
```

The `ip` node aggregates representations from all alerts involving it, then propagates a summary back. Alerts that are geometrically close in that IP neighbourhood learn similar embeddings — without any label telling the model "these alerts are part of the same campaign."

---

6. HGNN Architecture

6.1 Model Summary

```
MITREHeteroGNN(
    alert_feature_dim = 6,       # Base feature dimension
    hidden_dim        = 128,     # Embedding size throughout the network
    num_heads         = 4,       # GAT attention heads per relation
    num_layers        = 1,       # Single layer — empirically validated (see §13)
    dropout           = 0.3,
    num_clusters      = 10,      # Softmax head dim (unused in GAEC mode)
    aggr_method       = "mean",  # HeteroConv: how to aggregate across edge types
)
```

6.2 Input Encoders

Each node type has an independent encoder that projects its raw features into the shared 128-dim space:

```
alert        : CategoricalAlertEncoder or Linear(-1, 128)
user         : Linear(-1, 128)     # lazy init — dim inferred on first forward
host         : Linear(-1, 128)
ip           : Linear(-1, 128)
device       : Linear(-1, 128)
gateway      : Linear(-1, 128)
source_sensor: Linear(-1, 128)     # CS-2 sensor origin
process      : Linear(-1, 128)     # Linux APT
command_line : Linear(-1, 128)     # Linux APT
container    : Linear(-1, 128)     # Kubernetes
pod          : Linear(-1, 128)     # Kubernetes
```

`Linear(-1, 128)` is PyG's lazy initialisation — the actual input dimension is resolved on the first forward pass from the data. This allows the same model class to ingest any dataset schema without code changes.

6.3 GATConv Configuration (per Edge Type)

```
GATConv(
    in_channels  = 128,          # hidden_dim
    out_channels = 32,           # hidden_dim // num_heads
```

```
    heads        = 4,            # concat → output is 4 × 32 = 128
    dropout      = 0.3,
    add_self_loops = False,      # self-loops are redundant in the residual skip
)
```

Graph Attention: for each edge (i → j), the attention weight is:

```
α_ij = softmax( LeakyReLU( a^T [W·h_i || W·h_j] ) )
```

This means high-degree hub nodes (a common IP shared by 1,000 alerts) do not automatically dominate — the model learns to downweight noisy hub connections. GCN's fixed `1/√(deg_i · deg_j)` normalisation cannot do this.

6.4 Forward Pass — Step by Step

```
1. ENCODE  x_dict = { node_type: encoder(data[node_type].x) }
           # Each node type independently projected to 128-dim

2. SAVE    alert_raw = data['alert'].x
           # Kept for B1 input-side residual

3. FILTER  available_edges = { et: ei for et in data.edge_index_dict
                             if src_type in x_dict and dst_type in x_dict }
           # Gracefully skips absent node/edge types
           # → zero-shot cross-domain transfer without architecture changes

4. CONVOLVE  x_dict_new = HeteroConv(x_dict, available_edges)
             # Runs each edge type's GATConv independently
             # Aggregates multi-type messages with aggr="mean"

5. NORMALISE  for each node type k:
                x_dict[k] = LayerNorm( Dropout( ReLU( x_dict_new[k] ) ) )
                          + x_dict[k]           # ← residual skip connection

6. B1 RESIDUAL  x_dict['alert'] += alert_raw_proj(alert_raw)
                # Preserves raw feature variance — prevents information loss
                # during message passing on dense graphs

7. OUTPUT  cluster_logits = cluster_head(x_dict['alert'])   # [N_a, 10]
           return cluster_logits, x_dict
           # In GAEC mode: x_dict['alert'] (128-dim) is the embedding used for
clustering
           # cluster_logits (softmax head) is only used in UF refinement mode
(disabled)
```

6.5 The B1 Input Residual — Why It Exists

Without the input-side residual, message passing on a dense graph (e.g., a hub IP connected to 500 alerts) averages representations across all those alerts, destroying per-alert discriminative features. The B1 residual adds the raw 6-dim features (projected to 128-dim) back to the final alert embedding:

```
final_embedding = message_passing_output + alert_raw_proj(raw_6d_features)
```

This ensures that even if message passing collapses representations due to over-smoothing from high-degree neighbours, the raw feature signal remains in the final embedding. Empirically, this is critical for NSL-KDD (fully disconnected graph — message passing does nothing, but B1 ensures raw features flow through).

6.6 LayerNorm + Residual Skip

After each GATConv layer:

```
x_new = LN( Dropout( ReLU( GATConv(x) ) ) ) + x
```

LayerNorm stabilises training by normalising activations per layer. The residual skip ensures gradients flow to early layers even with non-zero dropout — standard practice from ResNet/Transformer architectures applied to the GNN setting.

6.7 Why One Layer?

With L=2 layers, an alert at distance 2 from a high-degree IP hub (which connects to thousands of unrelated alerts) receives averaged representations from the hub's entire neighbourhood. On UNSW-NB15, where 175K unique IPs create a star topology, this collapses embeddings: cosine_sim approaches 1.0, HDBSCAN finds a single cluster. Tested: L=1 gives ARI=0.40 on UNSW; L=2 gives ARI≈0.05 (embedding collapse). **L=1 is locked as the production default.**

---

7. Self-Supervised Training

7.1 Training Philosophy

No labels exist at training time. The model is trained to learn embeddings where **graph-connected alerts are similar** and **disconnected alerts are dissimilar**. The graph topology itself — which alerts share an IP, which are temporally adjacent — serves as the supervision signal.

7.2 Topological NT-Xent Loss

```
def topological_ntxent_loss(embeddings, graph, temperature=0.1, n_negatives=256):
    """
    For each connected alert pair in the graph → positive pair.
    Negatives: randomly sampled alerts from the same batch.
    """
    # Build adjacency from ALL alert-to-alert edge types
    adj = zeros(N, N)
```

```
    for et in [shares_ip, shares_host, temporal_near, semantic_similar]:
        adj[edge_index[0], edge_index[1]] = True

    pos_src, pos_dst = adj.nonzero()   # Up to 512 pairs sampled
    z = F.normalize(embeddings, dim=-1)

    # InfoNCE: pull positives together, push negatives apart
    pos_sim = (z[pos_src] * z[pos_dst]).sum(-1, keepdim=True) / temperature   #
[P, 1]
    neg_sim = bmm(z[neg_indices], z[pos_src].unsqueeze(-1)).squeeze() /
temperature  # [P, n_neg]
    logits  = cat([pos_sim, neg_sim], dim=1)   # [P, 1 + n_neg]
    loss    = cross_entropy(logits, zeros(P))   # positive always at index 0
```

Why topology > augmentation: SimCLR-style augmentation creates positive pairs by perturbing the same alert (adding noise, masking features). The topological approach uses *structural knowledge* — two alerts sharing a C2 IP address *should* be similar. This is a much stronger learning signal than random perturbation.

Temperature=0.1: Lower temperature → sharper cluster boundaries → better geometric separation in the embedding space. Empirically tuned; higher values (0.5) produced blurred clusters.

7.3 Dual Augmentation

Each training step applies two independent 15% edge dropouts to the same graph, creating two views (aug1, aug2). The total loss combines:

- Topological NT-Xent on aug1 (primary)
- Cross-graph alignment between aug1 and aug2 (optional, off by default)

Edge dropout at 15% prevents overfitting to specific graph structures while keeping enough topology for meaningful positive pairs.

7.4 Multi-Dataset Joint Training

```
NETWORK_IT_DATASETS = ['unsw_nb15', 'nsl_kdd', 'ton_iot', 'optc']
```

All four datasets contribute to every epoch. Each campaign is chunked into ≤500-alert graphs and pre-loaded to GPU. The model sees diverse graph structures, alert types, and tactic distributions each epoch — forcing it to learn features that generalise across network-IT domains rather than overfitting to a single dataset's topology.

Why joint training matters: A model trained only on UNSW-NB15 learns IXIA-specific traffic patterns. Joint training produces a backbone that understands network-layer attacks in general — enabling zero-shot transfer to TON_IoT (ARI=0.431) and NSL-KDD (ARI=0.752) without retraining.

7.5 Training Configuration (network_v9_v3)

Hyperparameter	Value	Rationale

Hyperparameter	Value	Rationale
Epochs	150	Convergence observed at ~120; extra epochs for stability
Batch size	8 graphs/step	GPU memory balance on RTX 5060 Ti
Hidden dim	128	Ablated: 64 underpowered, 256 no gain
Temperature (topo)	0.1	Sharper cluster boundaries
Temperature (aug)	0.5	Softer cross-view alignment
Optimizer	AdamW	lr=3e-4, weight_decay=1e-4
Scheduler	CosineAnnealingLR	T_max=150, smooth decay
Edge dropout	15%	Per augmentation view
num_layers	1	Over-smoothing prevention
Dropout	0.3	Standard regularisation
GPU	RTX 5060 Ti	~7s/epoch, ~17.5 min total
Seed	42	Full determinism (torch, numpy, random, HDBSCAN)

7.6 Checkpoint Evolution

Checkpoint	Date	Key change	Status
Pre-v7	Pre-Apr 14	MSE loss (wrong objective)	Collapsed — discarded
network_v7	Apr 14	NT-Xent fixed, 2 layers	Over-smoothing on UNSW
network_v9_v3	Apr 14	L=1, B1 residual, 6-dim	**Canonical production**
network_v9_v5	Apr (reverted)	15-dim contextual features	Catastrophic ARI collapse
network_cs_best	May 8	CS encoder fine-tune	Failed (AMI=0.0, early stop)

8. Inference & Clustering Pipeline

8.1 GAEC — Geometry-Aware Embedding Confidence

Standard confidence scoring uses softmax max-probability. This fails for out-of-distribution inputs: softmax always produces values summing to 1, so even a completely OOD alert gets a "confident" assignment.

GAEC replaces this with **HDBSCAN cluster membership probability**:

```
# After HDBSCAN clustering on alert embeddings:
membership_vectors = hdbscan.all_points_membership_vectors(clusterer)
# shape: [N_alerts, n_clusters]
# For each alert: soft probability distribution over ALL clusters
# (not just the assigned one — uses kernel density estimation)
```

```
confidence = membership_vectors.max(axis=1)
# Core cluster members    → confidence ≈ 1.0 (dense region)
# Border points           → confidence ≈ 0.3–0.7 (cluster edge)
# Noise / OOD points      → confidence ≈ 0.0–0.1
```

This is intrinsically calibrated to the embedding geometry — no temperature scaling required.

8.2 HDBSCAN Configuration

```
HDBSCAN(
    min_cluster_size = dataset-specific (5–50),
    min_samples      = min_cluster_size // 3,
    metric           = "cosine",     # semantic similarity
    algorithm        = "generic",    # required for cosine metric
    prediction_data  = True,         # enables all_points_membership_vectors()
    cluster_selection_method = "eom",
    cluster_selection_epsilon = 0.0–0.1,   # merges adjacent micro-clusters
)
```

Auto-tuning: if fewer than 2 clusters found, the system retries with progressively smaller `min_cluster_size` values [30, 20, 15, 10, 5], logging each attempt. This prevents complete failure on datasets with unusual density distributions.

cluster_selection_epsilon: controls post-hoc cluster merging. ε=0.1 on TON_IoT merges closely-related IoT attack subcategories (e.g., different DDoS variants) into coherent campaign groups without requiring k upfront.

8.3 Pre-Clustering Dimensionality Reduction

```
128-dim HGNN embeddings
    → PCA (128 → 16 components)          Always applied
    → [UMAP (16 → 10 components)]        Dataset-specific (off for TON_IoT)
    → [Soft-ZCA whitening]               Only for collapsed embeddings (SQTK_SIEM)
```

PCA: Removes linear noise and reduces HDBSCAN computational complexity. 16 components retain >95% explained variance in tested embeddings.

UMAP: Preserves local and global structure. Useful for UNSW-NB15 (global structure benefits spectral clustering). Disabled for TON_IoT (over-fragmentation: 10K sample with UMAP → 46 clusters; without → 37 stable clusters).

Soft-ZCA whitening: Applied when cosine_sim > 0.95 (moderate embedding collapse). Decorrelates the embedding matrix: `W = U(Λ + εI)^{-1/2} U^T`, then re-normalises to unit sphere. eps=0.1 is the production setting for SQTK_SIEM (cosine_sim 0.95 → 0.10).

8.4 Alternative Clustering Algorithms

Algorithm	When Used	Configuration
HDBSCAN	Default	Cosine metric, auto-tune, epsilon
Spectral	UNSW-NB15	k=8, cosine affinity, kmeans label assignment
BGMM	Experimental	max_components=30, full covariance
Prototype	Supervised mode only	SupervisedPrototypeHead loaded from checkpoint

8.5 Confidence-Gated UF Refinement (Disabled by Default)

The original architecture included a Union-Find fallback for low-confidence alerts. The design intent: when GAEC confidence < 0.6, re-correlate via rule-based Union-Find.

Empirical result: UF refinement is net-harmful across all datasets. Ablation (v2.6):

Dataset	With UF	Without UF	Explanation
UNSW-NB15	ARI=0.354	ARI=0.404	UF creates 100% singletons
NSL-KDD	ARI=0.217	ARI=0.257	Same pattern

Root cause: border points — alerts on the geometric boundary between two clusters — get low HDBSCAN confidence and trigger the UF path. UF assigns each one to a new singleton cluster based on exact entity matches, which almost never exist for border points. `use_uf_refinement=False` is the permanent production default.

---

9. Dataset Structural Analysis

Each dataset presents a fundamentally different graph structure. Understanding these differences is essential for interpreting why architectural decisions work on some datasets and fail on others.

9.1 UNSW-NB15 — Dense Network IDS with Protocol Diversity

Source: UNSW Cyber Range Lab, IXIA PerfectStorm traffic generator
Scale: 175,341 alerts · 48 campaign IDs · 9 attack categories
Real data: Partially synthetic (IXIA generator)

Raw schema:

```
src_ip, dst_ip, proto, service, src_bytes, dst_bytes, duration,
attack_cat (Fuzzers / Analysis / Backdoors / DoS / Exploits / Generic /
            Reconnaissance / Shellcode / Worms),
campaign_id (48 mapped campaigns)
```

Graph structure fingerprint:

- IP density: ~0.05 (medium) — many unique IPs from IXIA generation

- Temporal edges: YES — millisecond timestamps
- Host edges: minimal — lab environment
- Key structural problem: **campaigns 34 and 48 use IXIA fuzzing with 129 unique protocols** → protocol feature completely uninformative for those campaigns → diffuse embeddings HDBSCAN cannot compact

Architectural decisions and why:

Decision	Rationale
Spectral k=8 (not HDBSCAN)	Spectral uses global graph structure; we know k=8 campaign families exist. HDBSCAN finds density peaks and misses the global partition. 3.7× ARI improvement.
sample_size=2000 stratified	Full 175K alerts would make HDBSCAN intractable and dominate with majority-class Benign alerts
unsw_supcon_v7 checkpoint	SupCon fine-tuning with campaign labels; requires zero-padding 6→15 to match v9_v3 architecture
UMAP n_neighbors=30	Larger neighbourhood preserves global structure needed for spectral partitioning

SupCon fine-tuning journey:

- v1–v6 (all buggy): `finetune_supcon.py` used `alert_feature_dim=15` but `PublicDatasetGraphConverter` produces 6-dim features. The dimension mismatch caused silent weight corruption via `strict=False` loading. All v1–v6 checkpoints produce garbled embeddings.
- v7 (fixed): Zero-pad 6→15 before all HGNN forward calls. ARI (Spectral k=8) = 0.408.
- Result: SupCon v7 + Spectral matches zero-shot ceiling (0.538). **The IXIA protocol diversity (campaigns 34/48) is a hard structural ceiling for 6-dim features.**

Verified results:

Mode	AMI	ARI
Zero-shot (spectral k=8)	0.582	0.401
SupCon v7 (spectral k=8)	0.582	0.538
Supervised prototype	—	0.497

9.2 NSL-KDD — The Disconnected Graph

Source: KDD Cup 1999 (cleaned, de-duplicated by New Brunswick)
Scale: 125,973 alerts · 5 classes: DoS, R2L, U2R, Probe, Normal
Real data: No — synthetic traffic simulation

Raw schema:

```
duration, protocol_type, service, flag, src_bytes, dst_bytes,
land, wrong_fragment, urgent, hot, num_failed_logins, ...
(41 features) → mapped to 6 MITRE features
campaign_id = attack family (4 attack + 1 normal)
```

Graph structure fingerprint:

- **NO src_ip / dst_ip columns** — NSL-KDD does not include IP addresses
- **NO timestamps** — no temporal edges possible
- **Graph is fully disconnected** — the only edges are `semantic_similar` (same alert_type)
- Structural consequence: **HGNN ≈ MLP** — message passing has no graph to operate on

Why it outperforms supervised despite being a "broken" graph:

The 4 attack families are *linearly separable* in the 6-dim feature space:

```
DoS   : high src_bytes, TCP, specific flags, duration=0
R2L   : specific services (ftp, telnet), low src_bytes, many connections
U2R   : specific services (su, root shells), low byte counts
Probe : many unique dst_services, ICMP/UDP, short duration
```

The HGNN backbone, joint-trained on graph-rich datasets (UNSW, TON_IoT), learns a 128-dim projection where these distinct patterns have clear geometric separation. A linear GMM on raw 6-dim features (ARI=0.299) cannot exploit the non-linear geometry that the HGNN's learned projection reveals (ARI=0.752).

The **B1 input residual** is particularly important here: since message passing does nothing (disconnected graph), the final embedding is essentially `alert_raw_proj(6d_features) + zeros` — the backbone provides the projection, and raw features provide the signal.

Architectural decisions:

- No UMAP (no global structure to preserve in a disconnected graph)
- No sampling needed (125K is manageable; HDBSCAN on PCA-16 is fast)
- `num_layers=1` (already correct; matters especially here)
- `hdbscan_cluster_selection_epsilon=0.0` (compact, well-separated clusters)

Verified results:

Mode	AMI	ARI	Note
Zero-shot	**0.673**	**0.752**	Best unsupervised result
Supervised prototype	—	0.595	Labels hurt: adds noise to clean separability
Feature GMM baseline	—	0.299	2.5× gap from graph structure

Key interview point: "NSL-KDD proves that joint training transfers structural priors. The graph is disconnected, so HGNN ≈ MLP — yet it outperforms feature-only GMM by 2.5×. The backbone learned a

projection space where network attack families have geometric separation, and that learning transfers even when the target dataset has no graph structure."

9.3 TON_IoT — The Reproducibility Saga

Source: UNSW TON_IoT benchmark, IoT/IIoT sensor network
Scale: 211,043 alerts · 10 attack types (DDoS, DoS, backdoor, injection, MITM, password, ransomware, scanning, XSS, normal)
Real data: Partially real IoT sensor logs

Raw schema:

```
ts, src_ip, src_port, dst_ip, dst_port, proto, service, duration,
src_bytes, dst_bytes, conn_state, label (0/1), attack_type (10 classes)
```

Graph structure fingerprint:

- IP density: medium — IoT devices have fixed IPs, moderate connectivity
- Temporal edges: YES — Unix timestamps available
- IIoT topology: device/gateway edges present
- Key challenge: **full 211K dataset overwhelms HDBSCAN** (77 micro-clusters); needs stratified sampling

The Track 11 reproducibility crisis — this dataset had three compounding bugs that took the baseline from ARI=0.724 (historical) down to ARI=0.0076 (Track 8 final_metrics):

Bug 1 — HDBSCAN unseeded (`hgnn_correlation.py` lines ~1380, ~1395; `utils/clustering.py` line ~71):

```
# Before fix: random_state never passed → non-reproducible clustering
clusterer = HDBSCAN(min_cluster_size=..., metric="cosine", ...)

# After fix: seeded for cosine-compatible HDBSCAN (euclidean fallback)
if self.metric != "cosine":
    hdbscan_kwargs["random_state"] = self.seed
```

Bug 2 — Wrong feature dimension in fine-tune script (`training/finetune_cross_sensor.py` line 204):

```
# Before (wrong): v9_v5 experimental dim, never shipped
hgnn = MITREHeteroGNN(alert_feature_dim=15, ...)

# After (correct): matches network_v9_v3 training
hgnn = MITREHeteroGNN(alert_feature_dim=6, ...)
# Plus: zero-pad 6→15 before each forward pass
```

Bug 3 — sample_size dropped from DATASET_CONFIG (`experiments/run_gate_tuning.py`):

Parameter	Reference (013fcef)	After Track 7–10 edits	Effect
`sample_size`	10000	**MISSING**	Full 211K to HDBSCAN
`stratified_sample`	True	**MISSING**	Class imbalance
`use_umap`	True (then off)	True with extra params	Over-fragmentation

Fix progression: ARI 0.0076 → 0.109 (seeding) → **0.431** (config restoration)

Architectural decisions:

- `sample_size=10000, stratified_sample=True`: Critical. Stratification ensures all 10 attack types are represented. Full dataset overwhelms HDBSCAN with 77 micro-clusters.
- `use_umap=False`: Disabled — UMAP added fragmentation on this dataset (46 vs 37 clusters)
- `hdbscan_min_cluster_size=50`: Prevents IoT attack sub-variants from splitting
- `hdbscan_cluster_selection_epsilon=0.1`: Merges adjacent IoT attack micro-clusters

Verified results:

Mode	AMI	ARI	n_clusters
Zero-shot (Track 11)	**0.717**	**0.431**	37
Supervised prototype	—	**0.845**	10
Feature GMM baseline	—	0.233	—

9.4 DARPA OpTC — Perfect Binary Separation

Source: DARPA/SRI International, 2019 red team exercise on live enterprise
Scale: 4,656,650 events from Windows Sysmon across ~500 hosts
Real data: YES — live enterprise network
Campaigns: Binary — Benign vs RedTeam

Raw schema:

```
timestamp, hostname, process_name, action, src_ip, dst_ip, CampaignId,
object (file/network/registry), CommandLine (APT activity)
```

Graph structure fingerprint:

- Rich host graph: process + commandline + hostname edges all present
- Massive scale: 4.6M events → 10K stratified sample for evaluation
- Domain: **host-level telemetry** (Sysmon) vs checkpoint trained on network IDS alerts
- Binary structure: only 2 campaigns make standard ARI misleading

The ARI paradox:

```
Standard ARI = 0.048   ← looks bad
binary_ARI   = 1.000   ← perfect
cluster_purity = 0.999 ← 99.9% pure clusters

What happened: HGNN finds 25 sub-clusters.
Every RedTeam alert → a RedTeam sub-cluster.
Every Benign alert → a Benign sub-cluster.
ARI penalises having 25 clusters instead of 2.
binary_ARI collapses to: did each cluster go to the right campaign?
```

The 25 sub-clusters represent **meaningful attack phases**: C2 establishment, lateral movement, credential harvesting, exfiltration — distinct enough in embedding space to separate. A SOC analyst would call this a feature, not a bug.

Architectural decisions:

- `use_geometric_confidence=True` (GAEC): The `multidomain_v2` checkpoint has a softmax head trained on network labels — completely OOD for host telemetry. GAEC uses the raw embedding geometry, which transfers better.
- `checkpoint_override=network_v9_v3`: NOT multidomain_v2. network_v9_v3 was joint-trained on OpTC data — it has seen Sysmon-style events during training.
- `hdbscan_min_cluster_size=50`: At 10K sample, this prevents over-fragmentation
- `stratified_sample=True`: Ensures the 10K sample has both RedTeam and Benign events proportionally

Verified results:

Metric	Value	Interpretation
AMI	0.149	Limited by binary label space
ARI (standard)	0.048	Penalised for legitimate sub-clustering
binary_ARI	**1.000**	Perfect campaign-level separation
Purity	**0.999**	99.9% single-campaign clusters
Attack F1	0.942	Near-perfect attack/benign detection

9.5 SQTK_SIEM — Real Production SOC Data

Source: Real Security Operations Centre (anonymised)
Scale: 5,100 alerts from 7 distinct commercial sensors
Real data: YES — live enterprise production
Labels: `kcluster` (11 expert-defined campaign groups)

Multi-sensor composition:

Sensor	Count	Category

Sensor	Count	Category
Cisco	3,272 (64%)	Network/Firewall
F5 WAF	1,329 (26%)	Web App Firewall
Trend Micro	292 (6%)	Endpoint AV
Imperva	130 (3%)	Runtime App Protection
FireEye	42 (0.8%)	Next-Gen Firewall
Microsoft Defender	30 (0.6%)	EDR
Acalvio	5 (0.1%)	Deception Technology

Why this is the hardest dataset:

1. **Label quality**: `campaign_id` is 89% "UNKNOWN" — human analysts used tactic tags, not campaign IDs. `kcluster` is the expert-annotated grouping, but represents analyst judgment rather than ground truth.
2. **Multi-sensor schema heterogeneity**: Each sensor uses different field names, severity scales, and tactic vocabularies. Normalisation to MITRE format loses information.
3. **Severe class imbalance**: Cisco (3,272 alerts) vs Acalvio (5 alerts) — 654:1 ratio.
4. **Sparse graph**: Only 5,100 alerts → very few shared IPs/hosts → graph is nearly disconnected → HGNN cannot propagate meaningful signal.
5. **Embedding collapse**: Sparse graph + high-degree shared enterprise IPs → cosine_sim=0.91 (moderate collapse).

Architectural decisions:

- `siem_supcon_v4` checkpoint: Retrained with three fixes:
  - Fix 1: Label-pure edge filtering (cross-campaign edges removed during SupCon training)
  - Fix 2: Class-balanced SupCon loss (inverse-frequency weights for rare sensor types)
  - Fix 3: Alert feature enrichment (additional SIEM-specific features)
- `hdbscan_zca_whitening=True, eps=0.1`: Soft-ZCA mandatory — reduces cosine_sim 0.91→0.10
- `clustering_method="spectral", n_clusters=11`: HDBSCAN fails on sparse graphs with collapsed embeddings; spectral directly solves for the k=11 partition
- Label column: `kcluster` not `campaign_id` (89% UNKNOWN would make ARI meaningless)

Verified results:

Mode	AMI	ARI
Zero-shot + ZCA + Spectral	**0.342**	**0.184**
SupCon v3 + ZCA	—	0.174
Supervised prototype	—	0.053

Key insight: Prototype mode *underperforms* zero-shot on real SOC data because the expert `kcluster` labels are noisy — the prototype head learns the noise. Zero-shot clustering of the embedding geometry is more robust to label noise.

9.6 CICIDS2017 — Flow-Feature Benchmark

Source: Canadian Institute for Cybersecurity, 2017
Scale: 2.8M+ network flows, 14 attack classes
Real data: Partially real (lab environment with real tools)

Structure: Flow-level features only (78 statistical features). No IP/host entities in the standard format. The graph is sparse (temporal edges only). Best result: ARI=0.284 zero-shot, 0.440 with `use_burstiness=True` and `aggr_method=max`.

Limitation: CICIDS2017 was not part of the primary evaluation due to ambiguous label column mapping (fixed in Track 4). Results are preliminary.

---

10. Experiment History

Phase 1: Getting the Loss Right (Apr 2026)

Critical discovery: All pre-v7 training used **MSE loss** on embeddings. MSE minimises reconstruction error, which drives all embeddings toward the mean — perfect embedding collapse. NT-Xent pulls graph-connected pairs together and pushes disconnected pairs apart, maintaining geometric diversity.

```
v4 (MSE):     cosine_sim = 0.98–0.99   → clustering fails (1 cluster)
v9_v3 (NT-Xent): cosine_sim = 0.73–0.92 → clustering succeeds
```

This single loss change accounts for the majority of the performance improvement across all datasets.

Phase 2: Architecture Stabilisation (Apr 2026)

Version	Change	Finding
v2.1	Add confidence-gated UF	ARI improves, but UF creates singletons
v2.6	`use_uf_refinement=False` default	ARI 0.354→0.404 on UNSW
v2.7	singleton_fraction metric	Reveals UF creates 80%+ singletons universally
v2.9	soft_assign for border points	Border points ≠ noise; soft_assign ineffective

Phase 3: Multi-Dataset Joint Training (Apr 2026)

Version	Change	Finding
v2.10–v2.14	Joint training NSL+TON+UNSW	UNSW ceiling at 0.523 with SupCon
v2.15–v2.18	UNSW optimisation sweep	All failed — IXIA protocol diversity is structural
v2.19	TON_IoT zero-shot	ARI=0.737 (historical, later found unseeded)
v2.20	OpTC zero-shot	ARI=0.979 binary confirmed

Version	Change	Finding
v2.21	15-dim contextual features (v9_v5)	**Catastrophic**: ARI 0.714→0.003. Reverted.

v9_v5 failure analysis: Contextual features (IP frequency, temporal density) were computed from batch statistics — different at training vs inference time. The model learned to rely on batch-relative statistics, but at inference time those statistics have a different distribution. This is a fundamental train/inference distribution mismatch that cannot be fixed with fine-tuning.

Phase 4: HDBSCAN Bugs & Baselines (Apr 19, 2026)

Critical bug: HDBSCAN was called per-chunk (1K alert windows) instead of on all embeddings. With `min_cluster_size=50` on 1,000 samples = 5% threshold → 12 micro-clusters per window. Same config on 10K embeddings → 2 clusters.

Version	Fix	Impact
v2.24	Bridge edge ablation (ip→host)	Zero effect — closed
v2.25	Entity collapse ablation	Zero effect — closed
v2.26	HDBSCAN windowing fix	All prior OpTC ablations retroactively invalidated
v2.26	NSL-KDD feature baseline	HGNN 0.722 vs GMM 0.299 — graph value confirmed

Phase 5: Algorithm Additions (Apr 20–21, 2026)

Version	Addition	Key Result
v2.27	Spectral k=8 for UNSW	ARI 0.034→0.128 on zero-shot
v2.27	Soft-ZCA eps=0.1 for SQTK	cosine_sim 0.95→0.10
v2.27	SupCon dim-mismatch found	v1–v6 all corrupted (silent shape error)
v2.27	SupCon v7 fixed	ARI (Spectral) = 0.408 on UNSW
v2.29	Edge density cap (5 edges/node)	No improvement — over-smoothing from density is not the bottleneck

Phase 6: Prototype Integration (Apr 22–25, 2026)

Version	Action	Outcome
v2.31	Prototype training + integration	False cosine_sim > 1000 alarm
v2.32	Fix: similarity on embedding slice, not raw+embedding concat	True cosine_sim: 0.73–0.92 (normal)
v2.33	Prototype backbone fix — load HGNN from prototype checkpoint	TON_IoT: 0.2423→0.845
v2.34	Zero-shot regression: burstiness=False confirmed	Config standardised

Phase 7: Cross-Sensor Features (Apr 26–30, 2026)

CS-1 through CS-5 implemented:

- CS-1: `data_source` column in all converters
- CS-2: `source_sensor` node type + `collected_by`/`collects` edges
- CS-3: Kill-chain `precedes` temporal edges (2h window)
- CS-4: `MultiSourceIngestionPipeline` for N-sensor fusion
- CS-5: CLI flags for all CS features

18 tests passing. NSL-KDD ARI preserved at 0.7428. SQTK_SIEM: 7 source_sensor nodes confirmed.

CS-3 ablation: precedes=False gives ARI=0.139 vs precedes=True gives ARI=0.115. Kill-chain edges are noise, not signal. Closed.

Phase 8: Metrics Legitimacy (Track 8, Apr 30, 2026)

AMI promoted to primary metric. Added:

- Tactic Sequence Coherence (Kendall's τ vs ATT&CK kill-chain order)
- Attack F1 (binary attack/normal detection)
- Cluster purity

Track 8 sweep results (with the still-unseeded HDBSCAN — these numbers were later corrected):

Dataset	AMI	ARI
UNSW-NB15	0.582	0.401
NSL-KDD	0.673	0.752
TON_IoT	0.288	0.008
OpTC	0.149	0.048
SQTK_SIEM	0.342	0.184

Phase 9: TON_IoT Reproducibility (Track 11, May 8, 2026)

Three bugs found and fixed (see §9.3 for full analysis). Final verified baseline:

```
ARI: 0.0076 (Track 8, unseeded + wrong config)
  → 0.109  (seeding fixed)
  → 0.431  (sample_size + UMAP config restored)
```

---

11. Ablation Studies

11.1 UF Refinement — Net Harmful

Hypothesis: Low-confidence HGNN assignments can be improved by Union-Find rule-based correlation.

Result:

Dataset	UF Enabled	UF Disabled	Singleton Fraction (UF on)
UNSW-NB15	0.354	**0.404**	>0.80
NSL-KDD	0.217	**0.257**	>0.80

Root cause: Border points — alerts on the geometric boundary between two clusters — have low HDBSCAN membership probability and trigger the UF path. UF requires exact entity matches (shared IP, same host), which border points almost never have. Result: every border point becomes a singleton cluster. `use_uf_refinement=False` is permanent.

11.2 Bridge Edges (ip→host) — Zero Effect

Hypothesis: Adding `("ip", "resolves_to", "host")` and reverse edges would allow the model to correlate alerts from different sensors that share an IP↔hostname resolution.

Method: 3 paired seeds (42, 44, 45), OpTC, gate=0.65.

Result: ARI −0.0080 ± 0.0002 both conditions. Zero within-pair variance. Cohen's d=0.

Root cause: The existing `("alert", "shares_host", "alert")` edges already connect alerts sharing a hostname. The ip→host bridge adds an alternative path through the `ip` node — but the shortcut via `shares_host` is already present. Fully redundant. Closed.

11.3 Entity Collapse — Zero Effect

Hypothesis: Route alerts with resolvable IPs through their canonical host node (dual-edge: alert→ip AND alert→host), so alerts from different sensors describing the same machine share a common graph neighbour.

Result: Zero within-pair variance across 3 seeds. Same root cause as bridge edges: `shares_host` already provides the shortcut. Closed.

11.4 Kill-Chain Edges (CS-3) — No Improvement

Hypothesis: Temporal `precedes` edges connecting alerts within 2 hours would encode kill-chain ordering and improve campaign clustering.

Config	ARI	AMI
precedes=False	**0.139**	0.456
precedes=True	0.115	**0.459**

Root cause: IoT attack campaigns do not follow clean Recon→Exploit→Exfiltration chains. Different attack types (DDoS, backdoor, ransomware) often execute concurrently or in reverse order. Adding temporal ordering edges introduces structural noise. Closed.

11.5 v9_v5 Contextual Features — Catastrophic

Hypothesis: 15-dim features (6 base + IP frequency + temporal density) would improve embeddings by providing more context.

Result: NSL-KDD ARI 0.714→0.003. TON_IoT 0.737→0.028.

Root cause: Contextual features computed from batch statistics at inference time. The model learned batch-relative statistics at training time — a distribution mismatch. The 6-dim base features are dataset-independent and never batch-computed. Reverted. 6-dim is permanently locked.

11.6 UMAP Effect on TON_IoT

Config	n_clusters	ARI
Full 211K, no UMAP	77 (fragmented)	0.109
10K sample + UMAP (n_neighbors=30)	46	0.433
10K sample + UMAP disabled	37 (stable)	0.431

UMAP adds marginal cluster count without ARI gain for TON_IoT. Disabled.

11.7 SupCon v1–v6 Dim-Mismatch Bug

`finetune_supcon.py` initialised the HGNN with `alert_feature_dim=15` while `PublicDatasetGraphConverter` produces 6-dim features. With `strict=False` checkpoint loading, the mismatched `alert_raw_proj` weights were silently skipped — the model loaded with a randomly initialised input projection. All v1–v6 SupCon checkpoints produced corrupted embeddings.

v7 fix: `if graph['alert'].x.shape[1] < 15: graph['alert'].x = F.pad(...)` before each forward pass. This zero-pads 6→15 to match the checkpoint's expected input dimension while keeping the correct `alert_feature_dim=6` model initialisation.

---

12. Metrics Deep Dive

12.1 AMI — Adjusted Mutual Information (Primary)

```
MI(U,V)  = Σ_k Σ_j |U_k ∩ V_j| / N · log( N|U_k ∩ V_j| / (|U_k||V_j|) )
AMI(U,V) = ( MI(U,V) - E[MI] ) / ( max(H(U), H(V)) - E[MI] )
```

AMI measures how much information the predicted clustering shares with ground truth, adjusted for the information expected by chance for the given cluster counts. Range: [0, 1] (can be slightly negative).

Why AMI over ARI: ARI is pair-counting — it asks "for every pair of alerts, did we correctly say same-cluster or different-cluster?" This penalises sub-clustering: if the true labelling has 2 campaigns but HGNN finds 25 sub-clusters (OpTC), ARI=0.048 even if every pair within a true campaign is correctly grouped. AMI rewards information content: the 25 sub-clusters share nearly all information with the 2-campaign ground truth, so AMI=0.149 (limited by the binary label space, not by clustering quality).

12.2 ARI — Adjusted Rand Index (Secondary)

```
ARI = (RI - E[RI]) / (max(RI) - E[RI])
```

Pair-counting metric. Intuitive but penalises legitimate sub-clustering. Reported alongside AMI for comparability with published baselines. Range: [-1, 1].

12.3 GAEC Confidence

Per-alert confidence ∈ [0, 1] derived from HDBSCAN membership vectors. Reflects geometric density in the 128-dim embedding space:

- **Core cluster member** (high density neighbourhood): confidence ≈ 0.9–1.0
- **Border point** (cluster boundary): confidence ≈ 0.3–0.7
- **Noise point / OOD alert**: confidence ≈ 0.0–0.1

ECE measured at 0.015–0.022 across benchmarks (well-calibrated without temperature scaling).

12.4 Tactic Sequence Coherence

```
coherence_k = Kendall_τ(
    temporal_ordering_of_tactics_within_cluster_k,
    canonical_ATT&CK_ordering: [Recon=0, Resource_Dev=1, Initial_Access=2,
                                Execution=3, Persistence=4, Priv_Esc=5,
                                Defense_Evasion=6, Cred_Access=7,
                                Discovery=8, Lateral_Movement=9,
                                Collection=10, C2=11, Exfiltration=12,
                                Impact=13]
)
```

Measures whether alerts within each campaign cluster follow the expected kill-chain temporal order. Positive = correct ordering. Negative = reversed (common in IoT datasets where attack tools don't follow linear chains). NaN for datasets without timestamps (NSL-KDD).

12.5 Cluster Purity

```
purity = (1/N) · Σ_k max_j |cluster_k ∩ true_class_j|
```

For each cluster, what fraction is the most common true class? Purity=0.999 (OpTC) confirms that despite 25 sub-clusters instead of 2, each sub-cluster is internally homogeneous.

12.6 Attack F1

Binary F1 score for attack vs normal classification derived from cluster assignments. High values (0.969 TON_IoT, 0.942 OpTC) indicate the clustering cleanly separates malicious from benign traffic even without attack labels.

---

13. Design Decisions & Engineering Rationale

13.1 Why Heterogeneous Graph (Not Homogeneous)?

A homogeneous GNN treats all nodes identically. In the security domain, an IP address node and an alert node have fundamentally different semantics. The model needs to learn that "IP propagates neighbourhood similarity" differently from "user propagates ownership". HeteroGATConv gives each edge type its own learned weight matrix — a critical architectural requirement.

Empirical: HomogeneousGNN baseline achieves ~30% lower ARI on UNSW-NB15 by averaging across all entity types uniformly.

13.2 Why GAT Over GCN or GraphSAGE?

GCN: Fixed `1/√(deg_i · deg_j)` normalisation — treats all neighbours equally. A hub IP shared by 1,000 unrelated alerts would propagate a meaningless average to all of them.

GraphSAGE: Samples k neighbours per node — loses rare but important connections (e.g., a specific C2 IP shared by only 3 alerts).

GAT: Learns attention weights per edge — downweights noisy high-degree neighbours, emphasises rare but semantically important connections. Essential for the star-topology IP graphs in UNSW-NB15 and TON_IoT.

13.3 Why Self-Supervised (Not Supervised)?

MITRE-CORE targets production SOC deployment. Production constraints:

- Labelled campaign data is rarely available (analysts correlate *after* detection, not before)
- Labels are environment-specific and don't transfer across organisations
- Attack taxonomies evolve — a model trained on 2023 labels may not cluster 2024 TTPs correctly

Self-supervised training on graph topology labels nothing — it learns "similar graph structure = similar embedding" which is a universal principle across environments. The zero-shot transfer empirics confirm this: the model trained on UNSW-NB15 + NSL-KDD + TON_IoT + OpTC generalises to SQTK_SIEM (real SOC data) and CICIDS2017 without any retraining.

13.4 Why Topological NT-Xent Over SimCLR/MAE?

Method	Positive Pair Definition	Quality
SimCLR	Augmented views of same data point	Weak — random noise as supervision
MAE	Reconstructs masked node features	Pushes toward feature reconstruction, not cluster separation
Topological NT-Xent	Graph-connected alerts	**Strong — structural attack knowledge as supervision**

Two alerts connected by `shares_ip` genuinely *should* be similar. Two alerts connected by `temporal_near` genuinely *might* be related. Using the graph topology as the positive pair definition encodes domain

knowledge that random augmentation cannot.

13.5 Why HDBSCAN Over K-Means?

Property	K-Means	HDBSCAN
Requires k upfront	YES (unknown at deployment)	NO
Handles noise	NO	YES (noise = low-confidence alerts)
Variable-density clusters	NO (assumes spherical)	YES
Provides confidence	NO	YES (membership vectors)
Epsilon merging	NO	YES

In production deployment, the number of campaigns is unknown. HDBSCAN finds the natural cluster structure from the data geometry.

13.6 Why AMI as Primary Metric?

SOC analysts perform legitimate sub-campaign analysis. A 3-week APT campaign has 3 distinct phases — the HGNN correctly identifies these as 3 geometric clusters. ARI penalises this as error. AMI rewards it as information. The system should not be penalised for providing *more* detail than the ground truth label requires.

---

14. Interview Preparation

Elevator Pitch (30 seconds)

"MITRE-CORE is a purely unsupervised graph neural network that groups raw security alerts into attack campaigns. It models alerts, IPs, hosts, and users as a heterogeneous graph, trains via self-supervised contrastive learning on the graph topology, then clusters the resulting embeddings with HDBSCAN. No labels required — it generalises zero-shot across datasets, achieving 2.5× better clustering than feature-only methods even on structurally disconnected graphs."

Q: Walk me through the end-to-end architecture.

Raw alerts arrive as CSV or Parquet. They're normalised to a 16-column MITRE-format schema by the ingestion layer. The `dataset_profiler.py` fingerprints the data — IP density, has_timestamps, has_hostnames — and routes to the appropriate checkpoint and clustering config.

`AlertToGraphConverter` then builds a PyTorch Geometric `HeteroData` graph with up to 8 node types (alert, IP, host, user, device, gateway, process, command_line) and 29 edge relation types. Each alert gets a 6-dim integer feature vector encoding tactic, alert_type, hour, day_of_week, protocol, and service.

The `MITREHeteroGNN` runs a single-layer Heterogeneous Graph Attention Network. Each edge type has its own GATConv with 4 heads × 32-dim = 128-dim output. LayerNorm + residual skip after each layer. A B1 input-side residual adds the raw feature projection back to prevent information loss from message passing on dense graphs.

The resulting 128-dim alert embeddings go to the `EmbeddingConfidenceScorer`: PCA to 16 dims, optional UMAP, optional Soft-ZCA whitening, then HDBSCAN. The HDBSCAN membership vectors provide per-alert confidence scores — core cluster members get high confidence, noise/OOD alerts get near-zero.

Output: alert_id → campaign_id mapping with confidence scores. A SOC with 10,000 daily alerts sees ≤50 campaign narratives instead.

Q: Why does the model outperform GMM on NSL-KDD despite having no graph edges?

NSL-KDD is a disconnected graph — no IPs, no timestamps. HGNN is effectively an MLP on 6 features. Yet it achieves ARI=0.752 vs GMM's 0.299.

The explanation is in the joint training. The backbone was trained on UNSW-NB15, TON_IoT, and OpTC — datasets with rich graph structures. The topological NT-Xent loss teaches the model to project alerts into a 128-dim space where graph-connected alerts are geometrically close. This learned projection captures non-linear relationships between the 6 features that a linear GMM cannot exploit.

On NSL-KDD, the B1 input residual ensures the raw 6-dim features flow through to the final embedding. The backbone's learned projection space — shaped by graph-rich training — separates the 4 attack families (DoS, R2L, U2R, Probe) much more cleanly than GMM's Gaussian assumptions.

Q: What's the hardest engineering problem you solved?

The TON_IoT reproducibility crisis. A result that looked like ARI=0.737 turned out to be completely non-deterministic. Three bugs were operating simultaneously:

First, HDBSCAN was never given a random seed. The HDBSCAN library uses random initialisation for its minimum spanning tree — the same model, same data, same everything produced different cluster counts (24 vs 77) on different runs.

Second, in the cross-sensor fine-tuning script, I'd hardcoded `alert_feature_dim=15` (from a failed experiment) instead of 6. The checkpoint loaded with `strict=False`, so the mismatched `alert_raw_proj` weights were silently skipped — the model had a randomly initialised input projection. AMI=0.0 during training, no gradient flow through the backbone.

Third, `sample_size=10000` and `stratified_sample=True` had been dropped from the dataset config during Track 7-10 refactoring. HDBSCAN on the full 211K dataset fragmented into 77 micro-clusters instead of the reference 24.

Diagnosing this required git bisect to find the last commit where the result was stable, diffing the DATASET_CONFIG blocks across commits, inspecting checkpoint state dicts for missing/unexpected keys, and running diagnostic sweeps at each intermediate state. The fix progression was: ARI 0.008 → 0.109 (seeding) → 0.431 (config restoration).

Q: How do you handle different dataset schemas without retraining?

Three mechanisms:

Structural: `AlertToGraphConverter` builds only the edges that are possible from the available columns. If there's no `src_ip` column, no IP nodes are created. If there's no `timestamp`, no `temporal_near` edges are built. PyG's `HeteroConv` gracefully skips edge types absent from the current graph — `available_edges = {et: ei for et in data.edge_index_dict if et[0] in x_dict and et[2] in x_dict}`.

Feature-level: All datasets are normalised to the same 6-dim feature vector. `alert_type` is hashed to a fixed integer range; `tactic` is mapped from any string to the 0–13 MITRE ATT&CK integer. Datasets without timestamps get `hour=0` and `day_of_week=0`.

Routing: `dataset_profiler.py` selects not just the checkpoint but also the clustering config — whether to use Spectral or HDBSCAN, what `min_cluster_size` to use, whether UMAP should be applied. This per-dataset tuning is necessary: UNSW-NB15 needs Spectral k=8, TON_IoT needs HDBSCAN with UMAP disabled, SQTK_SIEM needs ZCA whitening.

Q: The model is "unsupervised" but you used SupCon fine-tuning — isn't that supervised?

Great question, and it's an important distinction.

The **core system is zero-shot unsupervised** — network_v9_v3 produces competitive results on every dataset without any labels. That's the primary contribution.

SupCon fine-tuning (semi-supervised mode) requires labelled campaigns for fine-tuning. We implemented it to answer: "how much can we gain if a small labelled set is available?" The answer for UNSW-NB15 is: essentially nothing — SupCon v7 + Spectral gives the same 0.538 ARI as zero-shot. The IXIA protocol diversity ceiling is structural, not a representation problem.

For SQTK_SIEM (real SOC data), SupCon actually *underperforms* zero-shot on the overall ARI because the `kcluster` labels are noisy expert judgements. The prototype mode (fully supervised) is even worse — it learns the label noise and collapses.

The practical deployment model: use network_v9_v3 zero-shot. If you have 500+ labelled campaigns from your specific environment, SupCon fine-tuning may help for datasets with structural complexity. For standard network-IT data, zero-shot is the production recommendation.

Q: What would you do differently?

Three things.

Embedding dimension per dataset: 128-dim is a good general purpose, but SQTK_SIEM (5,100 alerts) doesn't need the same capacity as UNSW-NB15 (175,341 alerts). A smaller backbone for smaller datasets would reduce embedding collapse risk.

Source_sensor_encoder lazy init: The CS-2 `source_sensor_encoder = Linear(-1, 128)` uses PyG's lazy init. When a checkpoint trained without CS-2 is loaded with `strict=False`, this encoder is randomly initialised and fires on the first forward pass with real data. The fix — explicit `Linear(feature_dim, 128)` with a shared constant — would require retraining but would make cross-sensor fine-tuning tractable.

Host-domain checkpoint: The network_v9_v3 checkpoint works remarkably well on OpTC (perfect binary separation) but only because OpTC has 2 campaigns. A dedicated checkpoint trained on Sysmon/EDR

telemetry with process + commandline features would genuinely unlock host-domain APT detection at campaign granularity.

Document version: May 8, 2026 | MITRE-CORE v2.39